

```

--
=====
=====
-- Music Distribution + Royalty Rail -- MVP Postgres
schema
-- Scope: distribution to DSPs (via white-label DaaS),
recording-royalty
-- collection, contributor splits, ETB payouts.
-- Money: always integer minor units (santim/cents) + ISO
4217 currency code.
-- Never floats for money.
-- Balances: derived from append-only ledger. No
mutable balance columns.
--
=====
=====

create extension if not exists "pgcrypto"; --
gen_random_uuid()
create extension if not exists "citext"; -- case-insensitive
email

-- -----
-- Shared helper: auto-maintain updated_at
-- -----

create or replace function set_updated_at() returns trigger
as $$
begin
    new.updated_at = now();
    return new;

```

```

end;
$$ language plpgsql;

-----

-- Enums
-----

create type user_role      as enum
('artist','label','staff','admin');
create type payee_type     as enum ('person','company');
create type kyc_status     as enum
('unverified','pending','verified','rejected');
create type payout_method_type as enum
('telebirr','chapa','cbe_birr','bank','other');

create type release_type   as enum
('single','ep','album','compilation');
create type release_status as enum
('draft','submitted','in_review','approved','delivering','live',
'update_pending','takedown_requested','taken_down','rejected');
create type track_status   as enum
('draft','ready','delivered','live','taken_down');

create type asset_kind     as enum ('audio','artwork');
create type asset_status   as enum
('uploaded','processing','ready','failed');

create type credit_role    as enum

('primary_artist','featured_artist','composer','lyricist','producer',
'mixer','arranger','performer','session_musician','remixer');

```

```

create type right_type      as enum
('recording','publishing'); -- MVP uses 'recording'
create type split_status    as enum
('pending','accepted','rejected');

create type delivery_action as enum
('deliver','update','takedown');
create type delivery_status as enum
('queued','sent','accepted','live','failed','taken_down');

create type validation_severity as enum ('error','warning');

create type report_source   as enum ('dsp','daas');
create type report_status   as enum
('imported','processing','processed','failed');
create type line_match_status as enum
('matched','unmatched','ignored');
create type sale_type       as enum
('stream','download','other');

create type ledger_entry_type as enum ('credit','debit');
create type ledger_source    as enum
('royalty','payout','adjustment','bonus','refund');

create type payout_status   as enum
('requested','approved','processing','paid','failed','cancelled');

create type charge_kind     as enum
('release_fee','subscription','addon');
create type charge_status   as enum
('pending','paid','failed','refunded','waived');
create type charge_provider as enum

```

```

('chapa','telebirr','manual');

create type identifier_type as enum ('isrc','upc');
create type identifier_origin as enum ('internal','daas');

-----
-- Reference: genres (DDEX genre mapping handled in
app layer)
-----

create table genres (
  id      uuid primary key default gen_random_uuid(),
  code    text unique not null,
  name    text not null,
  parent_id uuid references genres(id)
);

-----
-- App settings (min payout, hold days, default fee, etc.)
-----

create table app_settings (
  key      text primary key,
  value    jsonb not null,
  updated_at timestampz not null default now()
);
create trigger trg_app_settings_updated before update on
app_settings
for each row execute function set_updated_at();

-----
=====
=====
-- IDENTITY
-----
=====

```

=====

```
create table users (  
  id          uuid primary key default gen_random_uuid(),  
  email       citext unique not null,  
  role        user_role not null default 'artist',  
  password_hash text,          -- null if using external  
  auth  
  auth_provider text,          -- 'firebase','local', etc.  
  external_auth_id text,       -- e.g. firebase uid  
  full_name    text,  
  phone        text,  
  locale       text not null default 'en',  
  is_active    boolean not null default true,  
  created_at   timestampz not null default now(),  
  updated_at   timestampz not null default now(),  
  unique (auth_provider, external_auth_id)  
);  
create trigger trg_users_updated before update on users  
for each row execute function set_updated_at();
```

-- Public catalog identity. A user (esp. a label) may own many artists.

```
create table artists (  
  id          uuid primary key default gen_random_uuid(),  
  owner_user_id uuid not null references users(id) on  
delete restrict,  
  name        text not null,  
  bio         text,  
  country     char(2),         -- ISO 3166-1 alpha-2  
  spotify_artist_id text,      -- delivery matching  
  (avoids wrong-artist merges)  
  apple_artist_id text,  
  image_asset_id uuid,         -- FK added after
```

```

assets table
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now()
);
create index idx_artists_owner on artists(owner_user_id);
create trigger trg_artists_updated before update on artists
for each row execute function set_updated_at();

```

– Money identity: anyone who can hold a split and get paid.

– May be linked to a platform user, or be an external collaborator.

```

create table payees (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid references users(id) on delete set null, --
              null = external
  type        payee_type not null default 'person',
  display_name text not null,
  email       citext,
  country     char(2),
  kyc_status  kyc_status not null default 'unverified',
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);
create index idx_payees_user on payees(user_id);
create trigger trg_payees_updated before update on
payees
for each row execute function set_updated_at();

```

```

create table payout_methods (
  id          uuid primary key default gen_random_uuid(),
  payee_id    uuid not null references payees(id) on delete
              cascade,
  method      payout_method_type not null,

```

```

    details    jsonb not null,      -- {msisdn} telebirr,
    {account,bank} bank, etc.
    is_default boolean not null default false,
    verified   boolean not null default false,
    created_at timestampz not null default now()
);
create index idx_payout_methods_payee on
payout_methods(payee_id);
create unique index up_payout_default on
payout_methods(payee_id) where is_default;

--
=====
=====
-- ASSETS
--
=====
=====

create table assets (
    id          uuid primary key default gen_random_uuid(),
    owner_user_id uuid not null references users(id) on
delete restrict,
    kind        asset_kind not null,
    status       asset_status not null default 'uploaded',
    storage_key  text not null,      -- object storage path
    mime         text,
    bytes        bigint,
    checksum     text,              -- sha256 for
dedupe/integrity
-- image-only
width          int,
height         int,
-- audio-only

```

```

duration_ms int,
sample_rate int,
bit_depth int,
audio_format text,          -- 'wav','flac','aiff','mp3'
created_at timestampz not null default now()
);
create index idx_assets_owner on assets(owner_user_id);

-- deferred FK from artists.image_asset_id
alter table artists
add constraint fk_artists_image
foreign key (image_asset_id) references assets(id) on
delete set null;

--
=====
=====
-- CATALOG
--
=====
=====

create table releases (
id          uuid primary key default gen_random_uuid(),
owner_user_id  uuid not null references users(id) on
delete restrict,
primary_artist_id uuid not null references artists(id) on
delete restrict,
title        text not null,
version_title text,          -- e.g. "Deluxe",
"Remastered"
type         release_type not null,
status       release_status not null default 'draft',
display_artist text not null, -- exact string delivered

```


to DSPs

```
genre_id      uuid references genres(id),
language      text,          -- primary metadata
language (BCP-47)
release_date   date,
original_release_date date,    -- for re-releases
label_name     text,
copyright_c    text,          -- © line
copyright_p    text,          -- line
upc            text,          -- assigned at submit
artwork_asset_id uuid references assets(id) on delete
set null,
territories    text[] not null default '{}', -- 'WW' or ISO
alpha-2 list
explicit       boolean not null default false,
rejection_reason text,
submitted_at   timestampz,
created_at     timestampz not null default now(),
updated_at     timestampz not null default now()
);

create index idx_releases_owner on
releases(owner_user_id);
create index idx_releases_artist on
releases(primary_artist_id);
create index idx_releases_status on releases(status);
create unique index up_releases_upc on releases(upc)
where upc is not null;
create trigger trg_releases_updated before update on
releases
for each row execute function set_updated_at();

create table tracks (
id          uuid primary key default gen_random_uuid(),
release_id  uuid not null references releases(id) on
```

```

delete cascade,
track_number    int not null,
disc_number     int not null default 1,
title           text not null,
version_title   text,
status          track_status not null default 'draft',
primary_artist_id uuid references artists(id) on delete set
null, -- defaults to release artist
display_artist  text not null,
isrc            text,          -- assigned at submit
audio_asset_id  uuid references assets(id) on delete set
null,
genre_id        uuid references genres(id),
language        text,          -- lyrics language (BCP-47)
explicit        boolean not null default false,
is_instrumental boolean not null default false,
preview_start_ms int,
lyrics          text,
created_at      timestamptz not null default now(),
updated_at      timestamptz not null default now(),
unique (release_id, disc_number, track_number)
);
create index idx_tracks_release on tracks(release_id);
create unique index up_tracks_isrc on tracks(isrc) where
isrc is not null;
create trigger trg_tracks_updated before update on tracks
for each row execute function set_updated_at();

-- Credits = metadata delivered to DSPs (NOT money).
create table track_credits (
id          uuid primary key default gen_random_uuid(),
track_id    uuid not null references tracks(id) on delete
cascade,
name        text not null,

```

```

    role      credit_role not null,
    display_order int not null default 0
);
create index idx_track_credits_track on
track_credits(track_id);

--
=====
=====
-- SPLITS (money). Separate from credits.
-- Per (track, right_type): <=100% always (trigger), ==100%
at submit (app).
-- MVP populates right_type='recording' only.
--
=====
=====

create table track_splits (
    id      uuid primary key default gen_random_uuid(),
    track_id uuid not null references tracks(id) on delete
cascade,
    payee_id uuid not null references payees(id) on delete
restrict,
    right_type right_type not null default 'recording',
    role      credit_role,      -- optional context
    percent   numeric(7,4) not null check (percent > 0 and
percent <= 100),
    status    split_status not null default 'accepted',
    created_at timestamptz not null default now(),
    unique (track_id, payee_id, right_type)
);
create index idx_track_splits_track on
track_splits(track_id);
create index idx_track_splits_payee on

```

```
track_splits(payee_id);
```

– Enforce that splits never exceed 100% per (track,
right_type).

create or replace function enforce_split_sum() returns
trigger as \$\$

declare

 v_track uuid := coalesce(new.track_id, old.track_id);

 r record;

begin

 for r in

 select right_type, sum(percent) as total

 from track_splits

 where track_id = v_track

 group by right_type

 loop

 if r.total > 100 then

 raise exception

 'Split total % for track % (%) exceeds 100%', r.total,
v_track, r.right_type;

 end if;

 end loop;

 return null;

end;

\$\$ language plpgsql;

create constraint trigger trg_split_sum

 after insert or update or delete on track_splits

 deferrable initially immediate

 for each row execute function enforce_split_sum();

--

=====

=====

– IDENTIFIER ALLOCATION (atomic ISRC/UPC issuance)

–

=====

– One counter row per (type, prefix, year-bucket). UPC uses year_bucket=0.

```
create table identifier_counters (  
  identifier_type identifier_type not null,  
  prefix      text not null,      -- ISRC registrant (CCXXX)  
  or UPC company prefix  
  year_bucket  int not null,      -- ISRC 2-digit year; 0 for  
  UPC  
  last_sequence bigint not null default 0,  
  origin       identifier_origin not null default 'internal',  
  primary key (identifier_type, prefix, year_bucket)  
);
```

–

=====

– VALIDATION

–

=====

```
create table validation_issues (  
  id      uuid primary key default gen_random_uuid(),  
  release_id uuid not null references releases(id) on  
  delete cascade,  
  track_id  uuid references tracks(id) on delete cascade,  
  severity  validation_severity not null,  
  code      text not null,      -- e.g.  
  'SPLIT_TOTAL_NOT_100'
```

```

field      text,
message    text not null,
resolved   boolean not null default false,
created_at timestampz not null default now()
);
create index idx_validation_release on
validation_issues(release_id) where not resolved;

--
=====
=====
-- DISTRIBUTION / DELIVERY (via DaaS)
--
=====
=====

create table dsp_targets (
  id        uuid primary key default gen_random_uuid(),
  code      text unique not null, --
'spotify','apple','boomplay',...
  name      text not null,
  is_active boolean not null default true
);

create table deliveries (
  id          uuid primary key default gen_random_uuid(),
  release_id  uuid not null references releases(id) on
delete cascade,
  dsp_target_id uuid not null references dsp_targets(id)
on delete restrict,
  action      delivery_action not null,
  status      delivery_status not null default 'queued',
  ddex_message_id text,          -- ERN message id we
generated

```

```

    daas_delivery_id text,          -- id returned by the DaaS
backend
    dsp_release_url text,          -- live URL on the DSP
    error            text,
    queued_at        timestamptz not null default now(),
    sent_at          timestamptz,
    live_at          timestamptz,
    updated_at       timestamptz not null default now()
);
create index idx_deliveries_release on
deliveries(release_id);
create index idx_deliveries_dsp    on
deliveries(dsp_target_id);
create index idx_deliveries_status on deliveries(status);
create trigger trg_deliveries_updated before update on
deliveries
for each row execute function set_updated_at();

create table delivery_events (
    id          uuid primary key default gen_random_uuid(),
    delivery_id uuid not null references deliveries(id) on
delete cascade,
    status      delivery_status not null,
    note        text,
    created_at  timestamptz not null default now()
);
create index idx_delivery_events_delivery on
delivery_events(delivery_id);

--
=====
=====
-- ROYALTIES (ingestion -> matching -> allocation)
--

```

```
=====
=====
```

```
create table royalty_reports (
  id      uuid primary key default gen_random_uuid(),
  source   report_source not null,
  source_ref text,          -- DSP name / DaaS batch id
  period_start date not null,
  period_end date not null,
  currency char(3) not null, -- usually 'USD'
  filename text,
  status   report_status not null default 'imported',
  gross_minor bigint not null default 0,
  line_count int not null default 0,
  created_at timestamptz not null default now()
);
```

```
create table royalty_lines (
  id      uuid primary key default gen_random_uuid(),
  report_id uuid not null references royalty_reports(id)
on delete cascade,
  isrc     text,
  dsp_code text,
  territory char(2),
  sale_kind sale_type not null default 'stream',
  quantity  bigint not null default 0,
  revenue_minor bigint not null,    -- net payable to us, in
report currency
  currency char(3) not null,
  matched_track_id uuid references tracks(id) on delete
set null,
  match_status line_match_status not null default
'unmatched',
  created_at timestamptz not null default now()
```



```

);
create index idx_royalty_lines_report on
royalty_lines(report_id);
create index idx_royalty_lines_isrc on
royalty_lines(isrc);
create index idx_royalty_lines_track on
royalty_lines(matched_track_id);
create index idx_royalty_lines_status on
royalty_lines(match_status) where match_status =
'unmatched';

-- Explicit breakdown of a matched line across payees.
One credit per row.
create table royalty_allocations (
  id          uuid primary key default gen_random_uuid(),
  royalty_line_id uuid not null references royalty_lines(id)
on delete cascade,
  track_split_id uuid not null references track_splits(id)
on delete restrict,
  payee_id    uuid not null references payees(id) on
delete restrict,
  right_type   right_type not null,
  percent      numeric(7,4) not null,
  amount_minor bigint not null,    -- payee share, in line
currency
  currency     char(3) not null,
  created_at   timestamptz not null default now()
);
create index idx_royalty_alloc_line on
royalty_allocations(royalty_line_id);
create index idx_royalty_alloc_payee on
royalty_allocations(payee_id);

```

--

```

=====
=====
-- LEDGER (append-only) + BALANCE VIEW
--
=====
=====

create table ledger_entries (
  id          uuid primary key default gen_random_uuid(),
  payee_id    uuid not null references payees(id) on
delete restrict,
  entry_type  ledger_entry_type not null,
  amount_minor bigint not null check (amount_minor >=
0),
  currency    char(3) not null,
  source_type ledger_source not null,
  source_id   uuid,          -- royalty_allocation id /
payout id / etc.
  description text,
  created_at  timestampz not null default now()
);
create index idx_ledger_payee_ccy on
ledger_entries(payee_id, currency);
create index idx_ledger_source  on
ledger_entries(source_type, source_id);

-- Balance = credits - debits, per payee per currency.
create view v_payee_balance as
select
  payee_id,
  currency,
  sum(case when entry_type = 'credit' then amount_minor
else -amount_minor end) as balance_minor
from ledger_entries

```

```

group by payee_id, currency;

--
=====
=====
-- PAYOUTS (USD ledger -> ETB payment, FX recorded)
--
=====
=====

create table payouts (
  id          uuid primary key default gen_random_uuid(),
  payee_id    uuid not null references payees(id) on
delete restrict,
  payout_method_id uuid references payout_methods(id)
on delete set null,
  status      payout_status not null default 'requested',
  -- source (ledger) side
  source_currency char(3) not null,      -- 'USD'
  source_amount_minor bigint not null check
(source_amount_minor > 0),
  -- destination (paid) side
  currency     char(3) not null default 'ETB',
  fx_rate      numeric(18,6),           -- source->dest rate at
execution
  amount_minor  bigint,                  -- ETB paid, =
source*fx
  provider      payout_method_type,
  provider_ref  text,                    -- Telebirr/Chapa
transaction id
  failure_reason text,
  requested_at  timestampz not null default now(),
  paid_at      timestampz,
  updated_at    timestampz not null default now()

```

```

);
create index idx_payouts_payee on payouts(payee_id);
create index idx_payouts_status on payouts(status);
create trigger trg_payouts_updated before update on
payouts
for each row execute function set_updated_at();
-- On success: insert ledger_entries(entry_type='debit',
source_type='payout',
-- source_id=payout.id, amount=source_amount_minor,
currency=source_currency).

--
=====
=====
-- BILLING (your revenue — fees artists pay to distribute)
--
=====
=====

create table billing_charges (
  id      uuid primary key default gen_random_uuid(),
  user_id  uuid not null references users(id) on delete
restrict,
  release_id  uuid references releases(id) on delete set
null,
  kind      charge_kind not null,
  amount_minor bigint not null,
  currency  char(3) not null default 'ETB',
  status    charge_status not null default 'pending',
  provider  charge_provider not null,
  provider_ref text,          -- Chapa/Telebirr tx id
  created_at timestampz not null default now(),
  paid_at   timestampz
);

```

```

create index idx_billing_user on
billing_charges(user_id);
create index idx_billing_release on
billing_charges(release_id);
create index idx_billing_status on
billing_charges(status);

--
=====
=====

-- AUDIT LOG
--
=====
=====

create table audit_log (
  id      uuid primary key default gen_random_uuid(),
  actor_user_id uuid references users(id) on delete set
null,
  action   text not null,      --
'release.submit','payout.approve',...
  entity_type text not null,
  entity_id  uuid,
  meta      jsonb,
  created_at timestampz not null default now()
);
create index idx_audit_entity on audit_log(entity_type,
entity_id);
create index idx_audit_actor on audit_log(actor_user_id);

--
=====
=====

-- Seed defaults

```

```
--  
=====
```

insert into app_settings(key, value) values

('payout.min_minor',	'{"USD": 1000}'),	-- \$10.00
minimum		
('payout.hold_days',	'45'),	-- guard against
DSP clawbacks		
('billing.release_fee_minor_etb',	'150000')	-- 1,500 ETB
default per release		
on conflict (key) do nothing;		